Lab 1: Flipping an Array with CUDA

Phillip Pinter

February 22, 2024

1 Introduction

The mission of this lab session is to flip an array using CUDA with C++. This is done to illustrate how to write and execute parallel code on the GPU with CUDA. Flipping means that the elements of the arrays are now backwards.
For example: the array "[1, 2, 3, 4, 5]" becomes "[5, 4, 3, 2, 1]" after flipping it.

2 How to flip the array

To use CUDA, a kernal function needs to be made. When this function is called in the program, a specified number of blocks with each a specified number of threads is being made. These blocks of threads can run instructions in parallel. So in this exercise, each thread corresponds with an element in the array (assuming there are more threads available than there are elements) and since there is an input and output array, the threads won't conflict with each other so it is safe to run them all in parallel.

In the code on the next page, a kernel function is made. First, this function calculates the global index of the thread since there are multiple blocks with each a number of threads but these threads only have an local index of its block.
After calculating the index, the value of the element in the input array at that index is placed in the output array on the opposite side.

In the main fuction the following is done:

1. Define the variables. There are 2 kinds of arrays, a host and a device array. The host is your standard array stored in the host's memory and the device array is the array that is manipulated by the threads, stored in the device's (GPU's) memory. Memory is allocated and at the end freed for all these arrays.

2. Calculate the amount of blocks necessary to cover all elements of the array with a given block size.

3. The elements of the host input array is copied to the device input array. After calling the kernel function, the device output array is copied into the host output array.

4. Print the first and last 8 elements of the original input array and the flipped output array to verify the program worked.

```cpp
#include "cuda_runtime.h"
#include "device_launch_parameters.h"
#include <stdio.h>

// Kernel function to flip the elements of an array
__global__ void flipArray(int* input, int* output, int n)
{
    int idx = blockIdx.x * blockDim.x + threadIdx.x;
    if (idx < n)
        output[n - 1 - idx] = input[idx];
}

int main() {
    // Number of threads per block
    int const BLOCK_SIZE = 64;
    // Array size
    int const n = 1500;
    // Host arrays
    int* input, * output;
    // Device arrays
    int* d_input, * d_output;
    // Size of arrays for memory allocation
    int size = n * sizeof(int);

    // Allocate memory for host arrays
    input = (int*)malloc(size);
    output = (int*)malloc(size);

    // Initialize the host input array
    for (int i = 0; i < n; i++) {
        input[i] = i;
    }

    // Allocate memory for the device arrays
    cudaMalloc((void**)&d_input, size);
    cudaMalloc((void**)&d_output, size);

    // Calculate how may blocks are needed to go over all elements of the array
    // The block size is 256 and the array size is 1000, so there will be 4 blocks needed
    int numBlocks = (n + BLOCK_SIZE - 1) / BLOCK_SIZE;

    // Copy the host input array to the device input array
    cudaMemcpy(d_input, input, size, cudaMemcpyHostToDevice);

    // Launch the kernel with multiple blocks of size BLOCK_SIZE
    flipArray<<<numBlocks, BLOCK_SIZE>>>(d_input, d_output, n);

    // Copy device output array to the host output array
    cudaMemcpy(output, d_output, size, cudaMemcpyDeviceToHost);

    // Print the result (short version)
    printf("Input Array: \n");
    for (int i = 0; i < 8; i++)
        printf("%d ", input[i]);
    printf("\n...\n");
```

```
    for (int i = n - 8; i < n; i++)
        printf("%d ", input[i]);

    printf("\n\nOutput Array (Flipped): \n");
    for (int i = 0; i < 8; i++)
        printf("%d ", output[i]);
    printf("\n...\n");
    for (int i = n - 8; i < n; i++)
        printf("%d ", output[i]);

    printf("\n");

    // Free device memory
    cudaFree(d_input);
    cudaFree(d_output);

    // Free host memory
    free(input);
    free(output);

    return 0;
}
```
